Полное описание статического слоя, динамического слоя и Ghidra-интеграции

EH/Unwind Analyzer: что именно делает проект, что экспортируется в Ghidra, что входит в static/dynamic слой и где проходят границы ответственности

Папка проекта: /Users/alexander/Desktop/ВКР ГОТОВО/ВКР ПРОЕКТ ОКОНЧЕННЫЙ/ip_full
Папка документов: /Users/alexander/Desktop/ВКР ГОТОВО/ВКР ПРОЕКТ ОКОНЧЕННЫЙ/ip_full/docs/ru

1. Вывод после проверки docs/ru

В папке docs/ru уже есть материалы по проекту, но они решают разные задачи. Есть отдельная памятка по Ghidra, есть учебные книги, есть документы по признакам и отчетам. Однако единого документа, который одновременно и полно описывает Ghidra-интеграцию, и системно раскладывает static/dynamic слой по реализации, в папке не было. Поэтому этот PDF создан как отдельная техническая спецификация для защиты и передачи заказчику.

Документ	Что покрывает	Почему не закрывает вопрос полностью
EH_Unwind_Analyzer_Ghidra_Памятка_RU.pdf / .md	Практический workflow Ghidra: как создать CSV, импортировать, увидеть comments/bookmarks.	Это памятка по действиям, а не полное описание static/dynamic архитектуры и всех полей/ограничений.
как работает проект/Как_работает_проект_EH_Unwind_Analyzer.pdf	Объясняет общий pipeline проекта и связь PE -> RUNTIME_FUNCTION -> UNWIND_INFO -> findings -> reports -> Ghidra.	Материал широкий, но Ghidra и dynamic описаны обзорно; нет полного контракта по данным и действиям импортера.
EH_Unwind_Analyzer_Полное_руководство_по_признакам_и_эвристикам.pdf	Подробно раскрывает finding-коды, эвристики и смысл признаков.	Это не спецификация Ghidra-интеграции и не полный разбор dynamic collector.
КАК ЧИТАТЬ ОТЧЕТ_ПОЛНЫЙ.pdf	Помогает интерпретировать JSON/HTML отчет.	Не является архитектурным описанием слоев проекта.
РАБОТА С ГИДРОЙ/ФАЙЛЫ ДЛЯ ГИДРЫ/*	Содержит скрипты, CSV-разметку и готовые файлы для Ghidra.	Это артефакты и инструкция, а не единое доскональное описание механики.

2. Главная модель проекта

Проект состоит из трех логических слоев. Первый слой - статический: он читает PE x64 файл как байты и проверяет EH/unwind metadata без запуска target. Второй слой - динамический: он на Windows запускает target под Debug API и собирает runtime-события вокруг exception/unwind механизма. Третий слой - Ghidra-интеграция: он переносит адреса, диапазоны функций и комментарии в Ghidra, чтобы реверс-инженер вручную проверил найденные места.

```
PE file -> static parser -> RUNTIME_FUNCTION -> UNWIND_INFO -> heuristics -> JSON/HTML/CSV/Ghidra CSV -> manual validation in Ghidra
optional on Windows: --dynamic -> Debug API events -> dynamic_validation -> final assessment
```

3. Статический слой: что входит

Статический слой - основа проекта. Он не исполняет target и не зависит от того, может ли файл запуститься на текущей системе. Его задача - проверить, что записано в PE-файле на диске, и построить доказательную картину по EH/unwind metadata.

Этап	Что делает проект	Зачем это нужно
CLI/input	Принимает путь к PE и опции: --heuristics, --json, --html, --csv, --ghidra-csv, --dump, --ops, --strict-abi, --no-follow-chain, --max-chain-depth.	Задает режим анализа и формат результата.
Чтение файла	Открывает файл как bytes, считает metadata отчета и SHA-256 для воспроизводимости.	Позволяет потом доказать, какой именно бинарник анализировался.
PE parser	Проверяет MZ/PE, AMD64, PE32+, ImageBase, SizeOfImage, SizeOfHeaders, SectionAlignment, FileAlignment, секции и Data Directory.	Без корректной карты PE нельзя честно переводить RVA в file offset.
Секции	Проверяет raw/virtual выходы за границы, overlap, misalignment, права READ/WRITE/EXECUTE.	Ищет повреждение карты файла и подозрительное размещение metadata.
Exception Directory	Берет DataDirectory[IMAGE_DIRECTORY_ENTRY_EXCEPTION], проверяет RVA/Size, маппинг в файл, размер таблицы.	Это вход в .pdata и таблицу RUNTIME_FUNCTION.
RUNTIME_FUNCTION	Читает записи по 12 байт: BeginAddress, EndAddress, UnwindInfoAddress.	Получает диапазоны функций и указатель на UNWIND_INFO.
Проверки RUNTIME_FUNCTION	End > Begin, Begin/End внутри SizeOfImage, UnwindInfoRVA внутри image, сортировка, overlap, duplicate begin.	Автоматизирует ручной ритуал проверки .pdata.
UNWIND_INFO parser	Декодирует version, flags, prolog size, CountOfCodes, frame register/offset, unwind opcodes, EHANDLER/UHANDLER, CHAININFO.	Понимает внутреннюю структуру unwind metadata, а не просто печатает адреса.
Chain-follow	При включенном follow-chain проходит по chained unwind info до ограниченной глубины.	Нужен для chained function entries и защиты от циклов/глубоких цепочек.

Этап	Что делает проект	Зачем это нужно
Strict ABI	Опция --strict-abi фиксирует формальные ABI-предупреждения отдельно как unwind_abi_warnings.	ABI warning виден аналитику, но сам по себе не повышает score.
Heuristics	Собирает счетчики и findings по пяти семействам: table_integrity, unwind_integrity, metadata_placement, unwind_reuse, handler_dominance.	Переводит низкоуровневые факты в структурированные признаки.
Assessment	Считает score 0..100, suspicion, confidence, evidence_level, verdict, requires_manual_review, mitigation rules.	Дает triage-вывод, но не выносит автоматический приговор о вредоносности.
Reports	Пишет console, JSON, HTML, CSV и Ghidra CSV.	Разные форматы нужны для человека, автоматизации, таблиц и ручной проверки в Ghidra.

4. Что именно проверяет статический слой

Статический слой покрывает 27 finding-кодов. Они не являются случайным списком: они сгруппированы по тому, где в цепочке анализа возникла проблема.

Семейство	Finding-коды	Смысл
table_integrity	pe_section_raw_oob, pe_section_virtual_oob, pe_section_raw_overlap, pe_section_virtual_overlap, pe_section_misaligned, pe_exception_directory_oob, no_runtime_functions, invalid_ranges, rf_oob, unwind_rva_oob, rf_unsorted, rf_overlaps, duplicate_begin	Целостность PE-секций, Exception Directory и таблицы RUNTIME_FUNCTION.
unwind_integrity	unwind_parse_fail, unwind_anomalies, placement_integrity_combo	Корректность внутренней структуры UNWIND_INFO, flags, opcodes, operands, chain/handler data.
metadata_placement	unwind_in_exec_section, unwind_unusual_section, unwind_writable_section, handler_nonexec_section, handler_writable_section, placement_exec_nonexec_handler_combo, writable_metadata_handler_combo	Где лежат unwind metadata и handler targets, какие у секций права, нет ли странного размещения.
unwind_reuse	unwind_reuse_moderate, unwind_reuse_strong, reuse_dominance_combo	Массовое повторное использование одного UnwindInfoRVA многими функциями.
handler_dominance	handler_dominance	Один handler target доминирует у большого числа защищенных функций.

Важно: score - это инженерная оценка подозрительности, а не статистическая вероятность вредоносности. Confidence - уверенность в интерпретации набора свидетельств. Mitigation rules ослабляют confidence, если сигнал одиночный, sample маленький или возможна доброкачественная причина.

5. Что статический слой не делает

• Не запускает target и не видит события, которые появляются только во время исполнения.

• Не является полноценным дизассемблером и не доказывает семантику всего кода программы.

• Не сравнивает автоматически каждую машинную инструкцию пролога с каждым unwind opcode на уровне полного disassembly validation.

• Не объявляет файл malware. Он сообщает структурные свидетельства и формирует план ручной проверки.

• Не видит JIT/runtime function tables, если они не записаны в PE на диске; это зона dynamic layer.

6. Динамический слой: что входит

Динамический слой - Windows-only часть проекта. Он нужен не потому, что статика слабая, а потому что часть exception/unwind поведения может появляться только во время исполнения. Например, программа может зарегистрировать VEH/VCH/UEF или dynamic function table через RtlAddFunctionTable. В PE-файле на диске этого поведения может не быть.

Этап	Что делает проект	Что дает аналитику
Платформа	Работает только на Windows x64. На macOS/Linux возвращает controlled unsupported-result.	Честная граница: dynamic требует Windows Debug API.
Запуск target	Создает процесс через CreateProcessA с DEBUG_ONLY_THIS_PROCESS.	Target реально исполняется под наблюдением отладчика.
Module tracking	Обновляет список модулей процесса, считает observed modules, failures, truncations.	Позволяет переводить адреса breakpoint/events в module context.
API breakpoints	Ставит breakpoints на AddVectoredExceptionHandler, AddVectoredContinueHandler, SetUnhandledExceptionFilter, RtlAddFunctionTable, RtlInstallFunctionTableCallback.	Фиксирует runtime-регистрации exception handlers и dynamic function tables.
Static handler breakpoints	Берет handler_rva из статически разобранных UNWIND_INFO и ставит breakpoints на обнаруженные handlers.	Проверяет, дошло ли исполнение до handler, указанного в unwind metadata.
Custom handler breakpoints	После регистрации VEH/VCH/UEF ставит breakpoint на адрес зарегистрированного callback/handler.	Отделяет простой факт регистрации от реального handler hit.
Exception flow	Записывает EXCEPTION_DEBUG_EVENT, first-chance и second-chance события.	Показывает, возникали ли исключения и как target проходил exception dispatch.
Breakpoint handling	Снимает INT3, откатывает RIP, включает trap flag, re-arm через single step.	Позволяет наблюдать событие и не ломать нормальный поток target.
Trace	После handler hit может записать ограниченный RIP trace до trace_steps.	Дает короткий маршрут исполнения после попадания в handler.
Timeout/cleanup	Фиксирует timed_out, target_exited, exit code, instrumentation_complete.	Позволяет честно отличать полноценную сессию от неполной.

7. Какие runtime-сигналы дает dynamic layer

Группа	Поля/события	Значение
Состояние сессии	enabled, ran, ok, timed_out, target_exited, target_exit_code, instrumentation_complete	Была ли динамика включена, реально ли она запускалась, завершилась ли корректно.
Модули и breakpoints	modules_observed_peak, module_refresh_failures, breakpoint_set_attempts, breakpoint_set_failures, dropped_events	Насколько полно удалось поставить и поддерживать инструментирование.
Исключения	exceptions_seen, first_chance_seen, second_chance_seen, event type exception	Происходили ли исключения и на какой стадии они наблюдались.
UNWIND handlers	unwind_handlers_discovered, unwind_handler_bp_set, unwind_handler_hits, event type unwind_handler_hit	Дошло ли исполнение до handler, указанного в UNWIND_INFO.
VEH	veh_registrations, veh_hits, veh_registration, veh_handler_hit	Регистрировалась ли программа через AddVectoredExceptionHandler и получал ли VEH управление.
VCH	vch_registrations, vch_hits, vch_registration, vch_handler_hit	Регистрировался ли vectored continue handler и получал ли он управление.
UEF	uef_set_calls, uef_hits, uef_registration, uef_handler_hit	Устанавливался ли UnhandledExceptionFilter и срабатывал ли он.
Dynamic function tables	rtl_add_function_table_calls, rtl_install_function_table_cb_calls, rtl_add_function_table, rtl_install_function_table_callback	Появлялись ли runtime-регистрации unwind tables, типичные для JIT/runtime-generated code и защитных runtime-схем.
Trace	trace_count, trace[]	Ограниченный набор RIP-адресов после handler hit.

8. Как dynamic связан со static

Динамика не отменяет статический finding автоматически. Она добавляет контекст. Если static видит странное размещение unwind metadata, а dynamic видит handler hits или RtlAddFunctionTable, это может усилить подозрение или дать доброкачественное объяснение вроде JIT/runtime-generated code. Окончательный вывод делает реверс-инженер.

Ситуация	Как интерпретировать
Static finding есть, dynamic молчит	Статический факт остается. Возможно, путь исполнения не дошел до нужной логики, timeout мал, нужны аргументы, sample не запустился полноценно.
Static finding есть, dynamic видит VEH/VCH/UEF/handler hit	Подозрение может усилиться, потому что структурная странность связалась с runtime exception behavior.
Static finding есть, dynamic видит RtlAddFunctionTable	Появляется возможное объяснение через JIT/runtime-generated code или защитный runtime. Это не автоматическое оправдание, но важный контекст.

Ситуация	Как интерпретировать
Static чистая, dynamic видит handlers/function tables	Файл может быть статически корректен, но во время исполнения регистрировать runtime exception/unwind механизмы. Это отдельный runtime-сигнал.
Dynamic incomplete/timed out	Runtime evidence capped/ослабляется; нельзя делать сильный вывод по неполной сессии.

9. Что dynamic layer не делает

• Не является универсальной песочницей malware analysis.

• Не гарантирует покрытие всех веток target: он видит только то, что произошло за конкретный запуск и timeout.

• Не доказывает наличие JIT как технологии в целом; он фиксирует конкретные вызовы runtime function table API.

• Не заменяет статический анализ PE/EH/unwind metadata.

• Не работает полноценно на macOS/Linux, потому что опирается на Windows Debug API.

10. Ghidra-интеграция: зачем она нужна

Ghidra-интеграция нужна для ручной проверки. Analyzer находит адреса, диапазоны функций, unwind metadata, handler targets и anomalies. Ghidra показывает эти адреса в реальном дизассемблированном образе. Благодаря этому реверс-инженер не ищет вручную каждую RVA из JSON, а получает функции, комментарии и предупреждающие bookmarks прямо в CodeBrowser.

Главная формула: адрес в Ghidra = ImageBase + RVA. CSV хранит RVA, потому что RVA стабилен относительно image base, а Ghidra показывает virtual addresses в загруженной программе.

пример: ImageBase 0x140000000 + begin_rva 0x000013F0 = адрес Ghidra 0x1400013F0

11. Что пишет --ghidra-csv

Режим --ghidra-csv пишет компактный CSV, специально рассчитанный на импорт функций в Ghidra. В текущей реализации его основные колонки:

Колонка	Смысл
begin_rva	Начало диапазона функции из RUNTIME_FUNCTION.BeginAddress.
end_rva	Конец диапазона функции из RUNTIME_FUNCTION.EndAddress. Диапазон half-open: [begin, end).
name	Имя для функции, обычно sub_XXXXXXXX, где XXXXXXXX - BeginAddress.
comment	Собранный текст про UNWIND_INFO: version, flags, prolog, stack, frame, saved registers, handler_rva, chain, anomalies или parse failed.

Обычный CSV проекта шире: index, begin_rva, end_rva, size, unwind_rva, unwind_ver, unwind_flags, prolog_size, fixed_stack, frame_reg, frame_off, handler_rva, has_chain, anomalies, parse_error. Java importer умеет строить комментарий и из расширенных колонок, если name/comment отсутствуют; Python importer ожидает begin_rva, end_rva, name, comment.

12. Что попадает в Ghidra comment

Фрагмент comment	Откуда берется	Зачем нужен
UNWIND vN	UNWIND_INFO.Version	Показывает версию unwind metadata.
flags=...	UNWIND_INFO.Flags	Показывает EHANDLER/UHANDLER/CHAININFO.
prolog=N	SizeOfProlog	Помогает сравнить metadata с началом функции.
stack=NB	Декодированные UWOP_ALLOC/PUSH/SAVE effects	Ориентир по fixed stack size.
frame=REG+OFF	FrameRegister/FrameOffset	Показывает frame pointer модель.
saved_gp=[...]	Сохраненные nonvolatile GPR	Помогает понять, какие регистры должны восстанавливаться.
saved_xmm=[...]	Сохраненные XMM registers	Показывает сохранение SIMD-регистров.
handler_rva=...	EHANDLER/UHANDLER tail	Адрес handler target для ручной проверки.
chain_to=[begin..end] u=...	CHAININFO RUNTIME_FUNCTION	Показывает связанную unwind-запись.
anomalies=...	Флаги аномалий UNWIND_INFO	Сигнал для warning bookmark.
UNWIND parse failed: ...	Ошибка парсинга unwind structure	Сигнал для warning bookmark и ручной проверки.

13. Что делает Ghidra importer

Действие импортера	Подробно
Открывает CSV	Пользователь выбирает CSV вручную; Java-версия также может получить путь через args.
Проверяет колонки	Python-версия требует begin_rva, end_rva, name, comment. Java-версия требует begin_rva/end_rva и может сама собрать name/comment из расширенных колонок.
Переводит RVA в адреса Ghidra	Берет currentProgram.getImageBase().getOffset() и считает start = ImageBase + begin_rva, end = ImageBase + end_rva - 1.
Проверяет memory.contains	Если start/end вне памяти текущей программы Ghidra, строка пропускается.
Создает AddressSet	Диапазон функции становится телом функции в Ghidra.
Проверяет существующие функции	Если entry попадает внутрь другой функции или диапазон пересекает чужие функции, строка пропускается.
Создает функцию	Если функции на start еще нет, создает listing.createFunction/functionManager.createFunction с SourceType.IMPORTED.
Обновляет функцию	Если функция уже есть, может обновить body.
Переименовывает осторожно	Имя меняется только если источник имени DEFAULT или ANALYSIS. Ручные/пользовательские имена не перетираются грубо.
Добавляет pre-comment	Перед функцией добавляется строка с префиксом [EH/Unwind Analyzer]. Старые analyzer-комментарии заменяются, чужие комментарии сохраняются.
Ставит WARNING bookmark	Если comment содержит parse failed или anomalies=, ставится bookmark категории EH/Unwind Analyzer.

Действие импортера	Подробно
Работает в transaction	Все изменения делаются в транзакции Ghidra и коммитятся только при успешном проходе.
Печатает summary	В консоль выводятся created, updated, renamed, commented, bookmarks, skipped.

14. Что именно размечается в Ghidra

• Диапазоны функций из RUNTIME_FUNCTION: begin_rva/end_rva превращаются в функции Ghidra.

• Имена функций вида sub_XXXXXXXX, если имя можно безопасно поставить.

• Pre-comments с unwind-сводкой: версия, flags, prolog, stack, frame, saved registers, handler_rva, chain, anomalies/parse error.

• Warning bookmarks на функциях, где есть parse failed или anomalies=.

• Через комментарии аналитик видит handler_rva и chain_to, но сам handler как отдельная функция/структура автоматически не создается, если он не является строкой диапазона RUNTIME_FUNCTION.

15. Что Ghidra-интеграция не делает

• Не создает полноценные Ghidra data types для структуры UNWIND_INFO.

• Не размечает каждый байт .xdata как typed structure автоматически.

• Не исправляет дизассемблирование Ghidra и не патчит binary.

• Не выполняет dynamic analysis. Ghidra CSV - статический артефакт.

• Не заменяет ручную проверку: он подводит аналитика к адресам, которые надо открыть и осмыслить.

16. Как объяснять static/dynamic/Ghidra на защите

Вопрос комиссии	Короткий сильный ответ
Почему проект разделен на static и dynamic?	Static отвечает на вопрос, что записано в PE на диске. Dynamic отвечает на вопрос, что произошло при запуске вокруг exception/unwind. Это разные источники evidence.
Почему static больше dynamic?	Предмет проекта - структурные EH/unwind metadata PE x64. Эти данные в основном лежат в файле, поэтому основное покрытие статическое. Dynamic - слой подтверждения runtime-поведения.
Может ли dynamic отменить static finding?	Нет автоматически. Dynamic дает контекст: может усилить подозрение или показать доброкачественное объяснение вроде runtime/JIT function tables.
Зачем Ghidra, если есть HTML/JSON?	HTML/JSON показывают отчет. Ghidra показывает те же адреса в дизассемблированном образе, чтобы реверс-инженер руками проверил функцию, handler и section context.
Что именно проект делает для Ghidra?	Экспортирует begin/end RVA, имена и unwind-комментарии; importer создает/обновляет функции, добавляет pre-comments и warning bookmarks.
Это malware detector?	Нет. Это инструмент структурного анализа и triage EH/unwind-аномалий. Он помогает реверс-инженеру собрать evidence и быстрее перейти к ручной проверке.

17. Источники соответствия реализации

Файл проекта	Какая часть подтверждается
src/report.c	Форматы CSV/Ghidra CSV, build_unwind_comment, JSON/HTML reports, assessment/manual validation steps.
ghidra/import_unwind_csv.py	Python Ghidra importer: required columns, function creation/update, comments, bookmarks, overlap checks.
docs/ru/РАБОТА С ГИДРОЙ/ФАЙЛЫ ДЛЯ ГИДРЫ/ImportUnwindCsv.java	Java Ghidra importer: delimiter detection, fallback comment builder, safer import workflow.
include/dynamic.h	DynamicReport fields and declared defensive scope of dynamic collector.
src/dynamic_win.c	Windows Debug API implementation, API breakpoints, exception events, handler hits, trace.
include/heuristics.h	HeuristicReport fields, evidence families and counters.
docs/evidence_matrix.md	27 finding codes, scoring, confidence, mitigation rules, final case model.
src/pe.c / src/unwind.c	PE parsing, RVA mapping, RUNTIME_FUNCTION reading and UNWIND_INFO decoding.

18. Итоговая формулировка

Статический слой проекта автоматически проверяет структуру PE x64 EH/unwind metadata на диске. Динамический слой, доступный только на Windows x64, собирает runtime-свидетельства exception/unwind поведения: регистрации VEH/VCH/UEF, dynamic function tables, исключения, handler hits и короткий trace. Ghidra-интеграция переносит результаты статического анализа в CodeBrowser: создает/обновляет функции по RUNTIME_FUNCTION, добавляет

pre-comments и warning bookmarks, чтобы реверс-инженер быстро проверил конкретные адреса вручную. Вместе эти слои образуют доказательную цепочку, но не заменяют экспертный вывод.